



Rapport Technique :

Système de Gestion Morphologique Arabe

Modèle Racine-Schème

Réalisé par (1ING 3) :

Ahmed Chaabane

Firas Kahlaoui

Année Universitaire : 2025 – 2026

1 Introduction

L'objectif de ce projet est de concevoir un moteur performant de recherche, de génération et d'analyse morphologique pour la langue arabe, en exploitant le modèle linguistique "Racine-Schème". Ce document détaille l'architecture logicielle, les choix algorithmiques, les structures de données avancées mises en œuvre (Arbres AVL, Tables de hachage à indexation multiple), ainsi qu'une analyse exhaustive de leurs complexités temporelles et spatiales.

2 Architecture Logicielle et Technologies

Le projet adopte une architecture modulaire séparant rigoureusement la logique métier (Core) de l'interface utilisateur (GUI).

- **C++17 (Core)** : Choisi pour ses performances d'exécution et sa gestion fine de la mémoire (utilisation intensive des pointeurs intelligents `std::unique_ptr` et `std::shared_ptr`).
- **Qt6 Widgets (GUI)** : Framework robuste permettant de créer une interface de bureau réactive et multilingue grâce au paradigme des signaux et slots.
- **Node.js & Technologies Web** : Implémentation d'un serveur backend optionnel pour une visualisation interactive et déportée de l'arbre AVL via un navigateur.
- **CMake** : Système de build multiplateforme garantissant une compilation fluide et standardisée sous Linux et Windows.

3 Conception des Structures de Données

3.1 Arbre AVL : Indexation Dynamique du Lexique

Le choix de l'**Arbre AVL** (`AVLTree.h`) répond à la nécessité de maintenir un dictionnaire de racines trilitères et quadrilitères hautement performant en lecture.

- **Équilibrage Strict** : Contrairement aux arbres binaires de recherche classiques, l'AVL garantit que la différence de hauteur entre deux sous-arbres ne dépasse jamais 1 via des rotations (Gauche/Droite). Cela évite la dégénérescence en liste chaînée lors de l'insertion séquentielle de racines alphabétiques.
- **Stockage des Familles Morphologiques** : Chaque nœud encapsule une structure `std::vector<DerivedWord>`, permettant de lier une racine à l'ensemble des mots dérivés détectés dans le corpus, accompagnés de leurs fréquences d'apparition.

3.2 Table de Hachage à Multiples Index (Schèmes)

Pour la gestion des schèmes (`schemeHashTable.h`), nous avons implémenté une table de hachage à **chaînage séparé** utilisant trois niveaux d'indexation pour optimiser les requêtes selon le contexte d'exécution :

1. **Index par Nom** : Permet une récupération directe en $O(1)$ (ex : recherche du schème "maF3ouL").

2. **Index par Motif (Pattern)** : Utilisé pour vérifier l'existence préalable d'une structure morphologique lors de l'ajout de nouveaux schèmes.
3. **Index par Longueur** : Un tableau de listes (`lengthTable`) regroupant les schèmes par la taille du mot qu'ils génèrent. Lors de l'analyse d'un mot de 6 lettres, le moteur ne teste que les schèmes produisant des mots de 6 lettres, réduisant drastiquement l'espace de recherche.

4 Algorithmes et Moteur Morphologique

4.1 Génération Morphologique (Forward)

L'algorithme de génération agit comme une fonction de transformation $f(R, S) \rightarrow M$.

- **Processus d'Injection** : L'algorithme décompose le schème en caractères UTF-8. À chaque rencontre des marqueurs numériques ('1', '2', '3', '4'), il injecte la consonne radicale correspondante.
- **Traitement de la Gémiation (Shadda)** : Une étape de prétraitement (`expandShadda`) est appliquée. Si une Shadda (gémiation) est détectée, la consonne la précédant est dupliquée logiquement pour l'analyse, garantissant le respect des règles phonétiques arabes.

Exemple de Génération Morphologique

Racine : (ك ت ب)

Schème : (مفعول)

Résultat : (مكتوب)

4.2 Analyseur de Corpus et Apprentissage Automatique

L'analyseur (`CorpusAnalyzer.h`) traite des fichiers textes massifs pour en extraire des statistiques linguistiques. Pour chaque mot rencontré, il tente de procéder à une extraction inversée (Reverse Extraction) pour "deviner" la racine en soustrayant les affixes du schème.

- **Mode Strict vs Apprentissage** : En mode strict, seuls les mots correspondant à des racines connues sont comptabilisés. En mode apprentissage, si une structure correspond à un schème valide mais que la racine est absente de l'AVL, l'algorithme déduit et insère cette nouvelle racine, enrichissant ainsi le dictionnaire de manière autonome.

5 Analyse des Complexités

Nous distinguons la complexité temporelle (vitesse d'exécution) et la complexité spatiale (empreinte mémoire).

Opération	Meilleur Cas	Pire Cas	Justification
Recherche Racine (AVL)	$O(1)$	$O(\log n)$	L'arbre est strictement équilibré.
Insertion Racine (AVL)	$O(\log n)$	$O(\log n)$	Inclut le parcours et les rotations de rééquilibrage.
Recherche Schème	$O(1)$	$O(1 + \alpha)$	α représente le facteur de charge de la table.
Extraction de Racine	$O(1)$	$O(K_L \cdot L)$	K_L : nb de schèmes de longueur L , L : taille du mot.
Visualisation de l'Arbre	$O(n)$	$O(n)$	Parcours complet pour le dessin (Qt).
Analyse Corpus (N mots)	$O(N \cdot \log n)$	$O(N \cdot K_L \cdot \log n)$	Dépend fortement de la diversité des schèmes testés.

5.1 Complexité Temporelle

5.2 Complexité Spatiale

- **Arbre AVL** : $O(n + D)$ où n est le nombre de racines et D le nombre total de mots dérivés stockés. L'utilisation de pointeurs intelligents prévient les fuites de mémoire.
- **Table de Hachage** : $O(S)$ où S est le nombre de schèmes. Le chaînage séparé pour la résolution des collisions minimise la mémoire gaspillée par rapport à un adressage ouvert, particulièrement pertinent pour un dictionnaire de schèmes de taille modérée.

6 Défis Techniques et Solutions Apportées

1. **Sémantique et Encodage UTF-8** : La gestion des caractères arabes (qui occupent 2 octets par caractère en UTF-8) rend les fonctions standards C++ comme `std::string::size()` ou l'accès par index `str[i]` inopérantes.

Solution : Implémentation d'une classe utilitaire `StringUtils` capable de parser et d'itérer correctement sur les glyphes multi-octets.

2. **Rendu Graphique de l'Arbre AVL** : L'affichage d'un arbre contenant des milliers de nœuds dans l'interface Qt posait des problèmes de lisibilité et de performances.

Solution : Développement d'un composant `ZoomableView` intégrant des fonctionnalités de *Pan & Zoom*, couplé à un algorithme de calcul de coordonnées récursif dynamique pour éviter le chevauchement des sous-arbres.

7 Conclusion

Le système développé atteint un haut degré d'efficacité grâce à une synergie entre des structures de données spécialisées et une architecture logicielle robuste. L'indexation multiple des schèmes et l'équilibrage logarithmique des racines permettent de traiter des volumes de données importants (analyse de corpus) sans latence perceptible.